A1-15 spinlock + mutex – step by step

every line of code, and the theory behind it

How to use this. Do one step at a time: read it, type the code yourself (do not copy-paste – typing is how it sticks), check the "Done when" box, then go on. Before you start: `task start a1-15`. At the end: `task test`, done when **7 of 7** checks pass.

Type U – user space only. No kernel, no syscall number, no `case`: everything happens in the C library (`userspace/libc/`). The functions already exist as stubs that return `-1`.

Theory: `concepts.pdf` §12 atomic instructions, spinlocks and mutexes.

New to all of this? Read part 0 first.

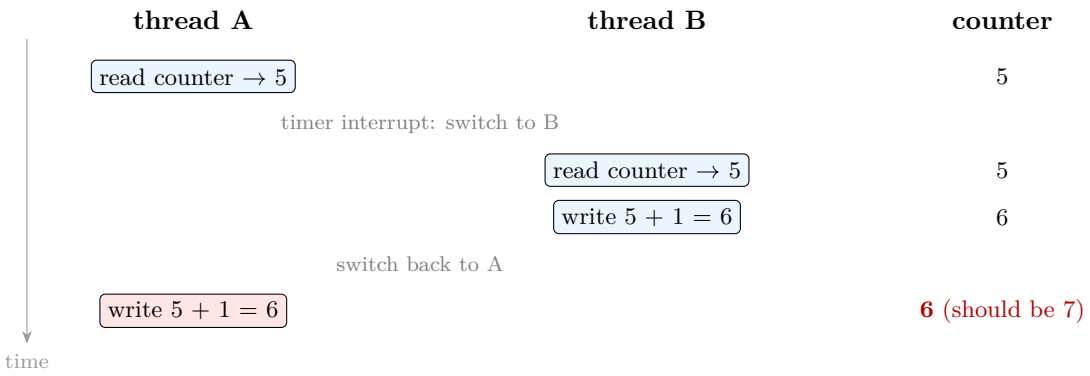
Part 0 – the basics, in plain words

The words

Word	What it means
Thread	One "worker" inside a running program. Several threads of one program run at the same time (or take turns very quickly) and share the program's memory, e.g. its global variables.
Shared variable	A variable that several threads read and write, like the counter in the test.
Time slice	The short time a thread may run before the operating system switches to another thread.
Timer interrupt	A signal from the hardware every few milliseconds. The kernel uses it to stop the running thread and let the next one run – at *any* instruction, without asking.
Race condition	A bug where the result depends on *when* the threads happen to be switched. Example: two threads both read the counter 5, both write 6 – one increment is lost.
Critical section	A piece of code that only one thread at a time may run, because it works on shared data (in the test: read the counter, wait a bit, write it back).
Lock	A variable that guards a critical section. `lock` = "wait until it is free, then take it". `unlock` = "give it back". Here a lock is *one* `unsigned int`: **0 = free, 1 = held**.
Spinlock	A lock where a waiting thread keeps *trying again* in a loop ("spinning") until the lock is free.
Mutex	"mutual exclusion" – the general name for a lock. A real mutex puts waiting threads to *sleep* in the kernel; in this task it may behave exactly like the spinlock.
`trylock`	Try *once*: take the lock if it is free, otherwise return at once with an error (no waiting).
Atomic	"Indivisible": an operation that happens in one step, so no other thread can come in between – not even a timer interrupt.
Test-and-set	The atomic operation of this task: "write 1 into the lock and tell me what was in it *before*" – in one step. Old value 0 = it was free and is now mine. Old value 1 = someone else has it. In C: `__sync_lock_test_and_set`; the CPU instruction is `xchg`.
`sched_yield`	A syscall: "I give up the rest of my time slice, let another thread run". The thread stays ready and continues later.

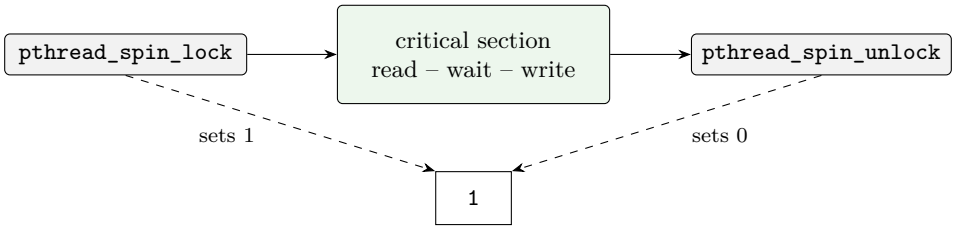
Word	What it means
libc	The C library every SWEB program uses (`printf`, `malloc`, `pthread_...`). It runs in user space, inside the program.
Stub	A function that exists but does nothing useful yet (here: `return -1;`).

Picture 1 – the problem: a lost increment (race condition)



Both threads did "+1", but the counter only grew by one. The test does this 80 000 times with 4 threads – without a working lock, many increments get lost.

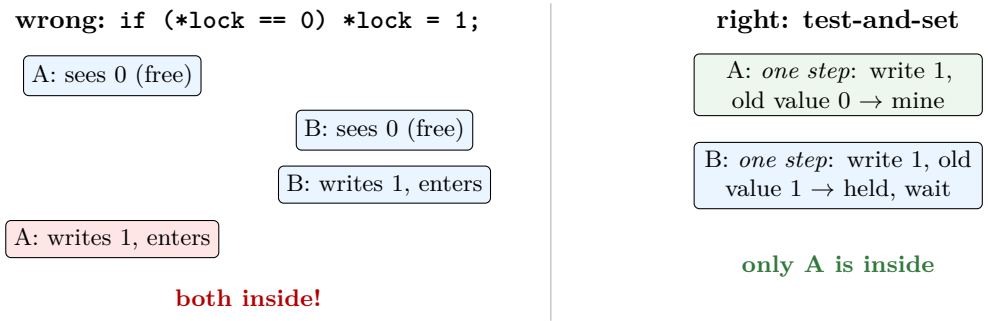
Picture 2 – the fix: a lock around the critical section



the lock (one `unsigned int`): 0 = free, 1 = held

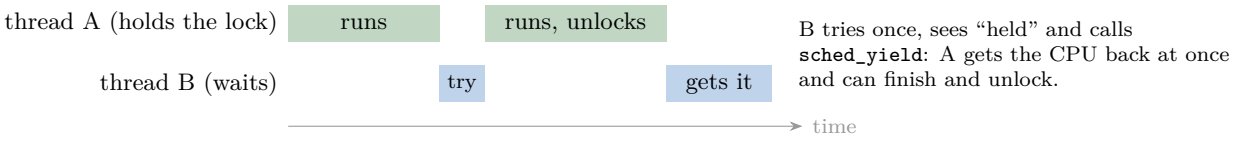
Only one thread at a time can be between `lock` and `unlock`. The others wait at `lock` until the holder calls `unlock`.

Picture 3 – why "check, then set" is not enough, and test-and-set is



On the left, a thread switch can happen *between* "look" and "write". On the right, looking and writing are one indivisible step – there is no "between".

Picture 4 – why wait with `sched_yield`



Without `sched_yield`, B would keep trying for its whole time slice – while A, the only thread that can unlock, does not get the CPU (SWEB runs on one CPU). That wastes time, it does not break the lock.

The goal (level 1)

```
Name:         pthread_spin_init/destroy/lock/trylock/unlock,
              pthread_mutex_init/destroy/lock/unlock (pthread.h - the stubs exist)
Description:  A lock is one unsigned int (0 = free, 1 = held). lock waits until the lock
              is free and takes it; trylock takes it only if it is free; unlock frees it.
              While waiting, give the CPU to the other threads (sched_yield) - the holder
              needs it to unlock. The mutex may behave exactly like the spinlock.
Return Value: 0, or -1 (NULL; trylock: the lock is held)
```

In your team's `pthread.h`: `typedef unsigned int pthread_spinlock_t;` and `typedef unsigned int pthread_mutex_t;` – both are just one number, so the mutex functions can simply call the spinlock functions.

The order: 8 steps, two files

#	What	File	Why
1	declare the 5 spinlock functions	`pthread.h`	they exist in `pthread.c` but are not declared – the test would not compile
2	include `sched.h`	`pthread.c`	for `sched_yield`
3	`pthread_spin_init`	`pthread.c`	a new lock starts free
4	`pthread_spin_destroy`	`pthread.c`	nothing to free
5	`pthread_spin_lock`	`pthread.c`	**the core: test-and-set + yield**
6	`pthread_spin_trylock`	`pthread.c`	test-and-set once
7	`pthread_spin_unlock`	`pthread.c`	set it back to 0
8	the four `pthread_mutex_...`	`pthread.c`	call the spinlock functions

1 Declare the spinlock functions `userspace/libc/include/pthread.h`

After the declaration of `pthread_cond_wait` (around line 61):

```
extern int pthread_spin_init(pthread_spinlock_t *lock, int pshared);
extern int pthread_spin_destroy(pthread_spinlock_t *lock);
extern int pthread_spin_lock(pthread_spinlock_t *lock);
extern int pthread_spin_trylock(pthread_spinlock_t *lock);
extern int pthread_spin_unlock(pthread_spinlock_t *lock);
```

- The functions are already *defined* (as stubs) in `pthread.c`, but the header does not *declare* them. The test includes `pthread.h`; SWEB compiles user programs with `-Werror=implicit-function-declaration` – calling an undeclared function is an error.
- Copy the signatures exactly from the stubs in `pthread.c` (a header and its `.c` must agree).
- `extern`: "this function exists somewhere else" – the usual style of this header.

Done when

Five declarations in `pthread.h`.

2 Include `sched.h` — `userspace/libc/src/pthread.c`

At the top, next to the other includes:

```
#include "sched.h"
```

`sched_yield` is declared in `sched.h`. Without the include, its call in step 5 is again an "implicit declaration" error.

Done when

One line at the top of `pthread.c`.

3 `pthread_spin_init` — `pthread.c`

Replace the body of the stub:

```
1 int pthread_spin_init(pthread_spinlock_t *lock, int pshared)
2 {
3   (void) pshared;
4   if (!lock)
5     return -1;
6   *lock = 0;
7   return 0;
8 }
```

Line 3. `pshared` ("shared between processes?") is not needed here. `(void)` tells the compiler (and the reader) "unused on purpose".

Lines 4–5. A `NULL` pointer is not a lock: return `-1` instead of crashing on `*lock`.

Line 6. `lock` is a *pointer* to the lock variable; `*lock` is the variable itself. A new lock starts **free** = 0.

Line 7. Success.

Done when

`init` sets the lock to 0.

4 `pthread_spin_destroy` — `pthread.c`

```
int pthread_spin_destroy(pthread_spinlock_t *lock)
{
  if (!lock)
    return -1;
  return 0;
}
```

The lock is one number inside the caller's memory – there is nothing to free. Only `NULL` is an error.

Done when

`destroy` returns 0 for a real lock.

5 `pthread_spin_lock` – the core — `pthread.c`

```
1  int pthread_spin_lock(pthread_spinlock_t *lock)
2  {
3    if (!lock)
4      return -1;
5    while (__sync_lock_test_and_set(lock, 1))
6      sched_yield();
7    return 0;
8  }
```

Lines 3–4. `NULL` check, as in `init` (the test checks it).

Line 5 – test-and-set. `__sync_lock_test_and_set(lock, 1)` is a compiler built-in. It writes 1 into `*lock` and returns the value that was there **before** – in **one indivisible step** (the CPU instruction `xchg`). So:

- returns 0 ⇒ the lock was free, and we just made it 1 ⇒ it is ours, the `while` ends;
- returns 1 ⇒ someone else holds it (writing 1 again changed nothing) ⇒ loop again.

Why not `while (*lock) {}` followed by `*lock = 1`? See picture 3: a thread switch between the look and the write lets two threads in.

Line 6 – `sched_yield`. The lock is held: give the CPU away so the holder can run and unlock (picture 4). Then the loop tries again.

Line 7. We hold the lock.

Trap

The test *cannot* prove that your lock is atomic: the naive "look, then write" version usually passes too, because the timer rarely hits exactly between the two instructions. Such bugs survive testing – in the interview the tutor asks *why* yours is correct. Answer: test-and-set is one indivisible step.

Done when

`lock` loops with test-and-set and yields while the lock is held.

6 `pthread_spin_trylock` `pthread.c`

```
1  int pthread_spin_trylock(pthread_spinlock_t *lock)
2  {
3    if (!lock)
4      return -1;
5    if (__sync_lock_test_and_set(lock, 1) != 0)
6      return -1;
7    return 0;
8  }
```

Line 5. The same test-and-set as in `lock`, but only **once**, no loop. Old value 1 ⇒ held by someone else ⇒ `-1` at once (line 6). We wrote 1 into a lock that was already 1 – nothing changed, so there is nothing to undo.

Line 7. Old value 0 ⇒ it was free and is ours now.

Done when

`trylock` succeeds on a free lock and fails at once on a held one.

7 pthread_spin_unlock — pthread.c

```
1 int pthread_spin_unlock(pthread_spinlock_t *lock)
2 {
3   if (!lock)
4     return -1;
5   __sync_lock_release(lock);
6   return 0;
7 }
```

Line 5. `__sync_lock_release` writes 0 = free. Why not just `*lock = 0`? The built-in also forbids the compiler (and CPU) to move writes of the critical section *after* the unlock – otherwise the next thread could see the lock free before the counter is written. (On x86, `*lock = 0` would usually work too; the built-in is the safe, portable way.)

Done when

`unlock` sets the lock back to 0.

8 The mutex functions — pthread.c

The card says the mutex may behave exactly like the spinlock. Both types are `unsigned int`, so each mutex function just calls its spinlock partner:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr)
{
  (void) attr;
  return pthread_spin_init(mutex, 0);
}

int pthread_mutex_destroy(pthread_mutex_t *mutex)  { return
    ↪ pthread_spin_destroy(mutex); }
int pthread_mutex_lock(pthread_mutex_t *mutex)     { return pthread_spin_lock(mutex); }
int pthread_mutex_unlock(pthread_mutex_t *mutex)   { return
    ↪ pthread_spin_unlock(mutex); }
```

- Write each one in the existing stub (keep the stubs' line layout – the short form above only saves space here). No duplicated logic: if the spinlock is right, the mutex is right.
- `attr` (mutex options) is not needed: `(void) attr;` marks it as unused on purpose.
- The `NULL` check comes for free: `pthread_mutex_lock(0)` calls `pthread_spin_lock(0)`, which returns `-1`.
- The mutex functions are *defined above* the spinlock functions in `pthread.c` – that is fine because step 1 declared them in `pthread.h`, which `pthread.c` includes.

Done when

`task test` shows **7 of 7 passed**.

The test – what each check means

Check	Which step makes it pass
init	steps 3 + 8 (`mutex_init` calls `spin_init`)
trylock on a free lock	step 6: old value 0 → 0
trylock on a held lock fails	step 6: old value 1 → `-1`
free again after unlock	step 7 sets 0, step 6 takes it again
spinlock: no increment lost	step 5: 4 threads × 20 000 increments = 80 000
mutex: no increment lost	step 8: the same with the mutex functions
NULL rejected	the `if (!lock)` in steps 5 and 8

Say it in three sentences

Remember

- A lock is one number: 0 = free, 1 = held; **lock** takes it, **unlock** sets it back to 0.
- Taking it must be **one indivisible step** (test-and-set = `xchg`): write 1, look at the old value – 0 means "mine".
- While it is held, **yield** the CPU so the holder can run and unlock.

Questions a tutor might ask

Question	Short answer
Why not `if (*lock == 0) *lock = 1;`?	a thread switch between the check and the write lets two threads in (picture 3)
What does `__sync_lock_test_and_set` return?	the *old* value; it writes the new one in the same step
Which CPU instruction is behind it?	`xchg` – exchange a register with memory, atomic on x86
Why `sched_yield` in the loop?	one CPU: the holder needs the CPU to unlock; spinning wastes the time slice
Does the test prove the lock is correct?	no – the naive version usually passes too; the argument is atomicity
Spinlock vs. real mutex?	spinlock: keeps trying; real mutex: sleeps in the kernel (wait queue), woken by unlock
When is spinning OK?	several CPUs and very short critical sections; on one CPU it only makes sense with yield
Why `__sync_lock_release` and not `*lock = 0`?	keeps the compiler/CPU from moving critical-section writes after the unlock
What happens if a thread calls `lock` twice?	it waits for itself forever – a deadlock (this lock is not recursive)
What about the kernel's locks?	the kernel has its own `SpinLock` and `Mutex` (`Mutex` sleeps) – user programs cannot use them directly